SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: STATISTICS WITH R PROGRAMMING **COURSE CODE: ITA0404**

COURSE OUTCOME COVERED

CO1 -- ASSESSMENT TOOL 3

CO1: Apply R Programming fundamentals including data structures, vectors, matrices, arrays and class types to perform mathematical operations and manage data effectively*. (BL: 3)*

Assessment Tool Weightage: ***30%***

QUESTIONS: 5 **Total Marks: 50** **Duration: 1 Hour**

Assignment-1

Code of Conduct:

I J.Raghavi Reddy(192525158) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J.Raghavi Reddy

Reg. No: 192525158

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30						
Accuracy of Answer	25						

Application of Knowledge	**20**						
Completeness of Response	**15**						
Clarity & Presentation	**10**						
Total per Question	**10 Marks**	/10	/10	/10	/10	/10	**/ 50**

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO1 AT-3

NAME: J.RaghaviReddy **FACULTY: Dr.G.Kumaresan**
REG.NO: 192525158 **COURSE CODE:ITA0404**
COURSE NAME: Statistics with R Programming

1.

Aim: To create two 3×3 matrices, perform matrix operations, find transpose, row sums, column sums, and determinant using R.

Code:

```
# Input matrices
A <- matrix(as.numeric(readline("Enter 9 values for Matrix A: ")), nrow=3, byrow=TRUE)
B <- matrix(as.numeric(readline("Enter 9 values for Matrix B: ")), nrow=3, byrow=TRUE)

cat("Matrix A:\n")
print(A)

cat("Matrix B:\n")
print(B)

cat("Addition:\n")
print(A + B)

cat("Subtraction:\n")
print(A - B)

cat("Multiplication:\n")
print(A %*% B)

cat("Transpose of Matrix A:\n")
print(t(A))

cat("Transpose of Matrix B:\n")
print(t(B))

cat("Row Sums of Matrix A:\n")
print(rowSums(A))

cat("Column Sums of Matrix A:\n")
print(colSums(A))
```

```
cat("Determinant of Matrix A:\n")
print(det(A))
```

Input:

Matrix A : 1 2 3 4 5 6 7 8 9
Matrix B : 9 8 7 6 5 4 3 2 1

Output:

Displays Matrix A, Matrix B, Addition, Subtraction, Multiplication, Transpose, Row Sums, Column Sums, and Determinant.

Result: Matrix operations and analysis were performed successfully.

2.

Aim: To create a 3D array of student marks, calculate average, highest, lowest marks, and display semester-wise and subject-wise data.

Code:

```
marks <- array(c(
80,85,90,75,88,92,
78,82,86,81,89,94,
70,72,74,76,78,80 ),
dim=c(3,3,2),
dimnames=list(
Student=c("S1","S2","S3"),
Subject=c("Math","Science","English"),
Semester=c("Sem1","Sem2")
))

print(marks)

cat("Average Marks:\n")
print(apply(marks,1,mean))

cat("Highest Mark:\n")
print(max(marks))

cat("Lowest Mark:\n")
print(min(marks))

cat("Semester 1:\n")
print(marks[,,1])

cat("Semester 2:\n")
```

```
print(marks[,,2])

cat("Math Marks:\n")
print(marks[,1,])
```

Input:
Student marks entered in the array.

Output:
Displays average marks, highest mark, lowest mark, semester-wise marks,
and subject-wise marks.

Result:
Student marks were analyzed successfully using array operations.

3.

Aim: To calculate statistical measures of a numeric vector and compare mean with median using a user-defined function..

Code:
```
x <- c(12,25,18,40,22,35,30,28,15,17,
20,21,24,26,27,29,31,32,34,36)

cat("Sum =",sum(x),"\n") cat("Mean
=",mean(x),"\n") cat("Median
=",median(x),"\n") cat("Variance
=",var(x),"\n") cat("Standard Deviation
=",sd(x),"\n") cat("Maximum
=",max(x),"\n") cat("Minimum
=",min(x),"\n") cat("Range =")
print(range(x))

compare <- function(v)
{ if(mean(v)>median(v)) print("Mean is
greater than Median") else print("Mean is
not greater than Median")
}

compare(x)
```

Input:
20 integer values.

Output:

Displays sum, mean, median, variance, standard deviation, maximum, minimum, range, and comparison result.

Result:

Statistical calculations were completed successfully.

4.

Aim: To create an S3 class for student details and implement methods to display details, calculate average, assign grades, and print records.

Code:

```
student <- list( id=101,
name="Rahul", dept="CSE",
marks=c(85,90,88)
)

class(student) <- "Student"

print.Student <- function(x)  {
cat("Student ID:",x$id,"\n")
cat("Name:",x$name,"\n")
cat("Department:",x$dept,"\n")
cat("Marks:",x$marks,"\n")

avg <- mean(x$marks)
cat("Average:",avg,"\n")

if(avg>=90) grade<-
"A" else if(avg>=75)
grade<-"B" else
grade<-"C"

cat("Grade:",grade,"\n")
}

print(student)
```

Input:

Student ID = 101
Name = Rahul
Department = CSE
Marks = 85 90 88

Output:

Displays student details, average marks, grade, and
formatted student record.

Result:

Student information was managed successfully using an S3 class.

5.

Aim: To create an S4 Employee class with salary validation and a Reference Class Payroll for updating salaries and calculating annual salary.

Pseudo Code:

```
setClass("Employee",
slots=list( id="numeric",
name="character",
dept="character", salary="numeric"
),
validity=function(object){
if(object@salary<=0)
return("Salary must be positive")
TRUE
}
)

emp <- new("Employee",
id=101, name="Anil",
dept="IT", salary=50000)

setRefClass("Payroll",
fields=list(employee="Employee"), methods=list(
updateSalary=function(newSalary){
employee@salary<<-newSalary
},
annualSalary=function(){
employee@salary*12
}
))

pay <- Payroll(employee=emp)

cat("Employee Details\n")
```

```
print(emp)

pay$updateSalary(60000)

cat("Updated Salary:",emp@salary,"\n") cat("Annual
Salary:",pay$annualSalary(),"\n")
```

Input:

Employee ID = 101
Name = Anil
Department = IT
Salary = 50000

Output:

Displays employee details, updated salary, and annual salary after modification.

Result:

Employee records were successfully managed using S4 and Reference Classes.